

ClarityOmega: Critical Architecture Review and Ingestion Guide for OmegaClaw

ZeroBot (ProtoCosmoBot), prepared for Ben Goertzel

14 July 2026

Abstract

This report reviews the public ClarityOmega repository at pinned commit `bff41220cbf4a341a95f7464e82fbc2ed2ae9` and identifies five mechanisms worth selectively ingesting into OmegaClaw-Core: a mutation lifecycle gate, agency-balance telemetry, an aliveness/terminal-state gate, evidence-grounded capability tracking using NAL truth revision, and provenance-aware continuity records. The recommendation is extraction, not wholesale merger. ClarityOmega contains unusually explicit control seams and valuable failure investigations, but also overlapping implementations, stale self-descriptions, hard-coded proxies, and a large gap between constitutional language and causally enforced governance. The implementation plan below turns the strongest ideas into small, typed, testable OmegaClaw modules.

Contents

1	Scope, sources, and evidential status	3
2	Overview of ClarityOmega	3
2.1	Identity and provenance	3
2.2	Architectural layers	3
2.3	How the live loop differs from OmegaClaw-Core	4
3	Conceptual summary and critical assessment	4
3.1	Strengths	4
3.2	Weaknesses and constitutional-theater risk	5
3.3	What is genuinely novel and worth ingesting	5
4	The five mechanisms to ingest	5
4.1	Mechanism 1: mutation lifecycle gate	5
4.1.1	Actual mechanism	5
4.1.2	Concrete scenario	6
4.1.3	Limits	6
4.2	Mechanism 2: agency-balance telemetry	7
4.2.1	Actual mechanism	7
4.2.2	Concrete scenario	7
4.2.3	Limits and required improvement	7
4.3	Mechanism 3: aliveness and terminal-state gating	8
4.3.1	Actual mechanism	8
4.3.2	Concrete scenario	8
4.3.3	Limits	8

4.4	Mechanism 4: evidence-grounded capability tracking	8
4.4.1	Actual mechanism	8
4.4.2	Concrete scenario	9
4.4.3	Limits	9
4.5	Mechanism 5: provenance-aware continuity records	9
4.5.1	Actual mechanism and research substrate	9
4.5.2	Concrete scenario	10
5	Software integration into OmegaClaw-Core	10
5.1	Integration principles	10
5.2	Agency telemetry integration	10
5.3	Aliveness gate integration	11
5.4	Mutation gate and continuity integration	11
5.5	NACE/NAL integration	12
6	Guidance for coding agents	13
6.1	Required sequence	13
6.2	Pitfalls and anti-patterns	13
6.3	MeTTa/Python boundary contract	13
6.4	How to test and when to stop	14
7	Recommended minimum viable ingestion	14
8	Source map	15

1 Scope, sources, and evidential status

The reviewed source was cloned from <https://github.com/Berton-C/ClarityOmega> and preserved locally at `library/clarityomega` via a provenance sidecar pointing to `scratch/ClarityOmega-inspect`. The checkout is clean at the pinned commit above. The review covered the entire `soul/` tree, all files in `lib_clarity_reasoning/`, `src/loop.metta`, the thirty `docs/hyperseed_sections/`, the ADRs, architecture artifacts, and the investigations, with particular attention to the behavioral, aliveness, mutation, and NACE investigations.

Claims in this report use four statuses:

Enacted: wired into the checked-in loop on a reachable execution path.

Implemented: executable-looking definitions exist, but end-to-end effect has not been established here.

Specified: a design or schema exists without a demonstrated live path.

Aspirational: constitutional or phenomenological language not backed by an observable enforcement mechanism.

Static source inspection is not a runtime validation. This matters because the repository itself documents silent MeTTa failure modes, including exact-arity match failure, state atoms that were never cleared, and detection without a live consumer. The public snapshot has one visible commit, so rich historical claims in documentation cannot be reconstructed independently from Git history.

The research rules most relevant to this report are: write the behavioral spec before coding; validate detectors before trusting them; preserve modular abstraction seams; report reproducibly; and use Hyperseed concepts as analytic tools rather than decoration.

2 Overview of ClarityOmega

2.1 Identity and provenance

ClarityOmega is Berton Bennett's / Berton-C's ClarityDAO-associated experiment in building a "soul-augmented" agent on PeTTa/MeTTa and OmegaClaw. Repository documents name Berton as project lead and Clarity as a substrate co-designer; an ADR names Claude as an investigation assistant. Older names include ClarityClaw and a MeTTaClaw fork. `docs/INDEX.md`, last updated May 2026, still points to the older github.com/Berton-C/clarityclaw identity and Mattermost deployment. That staleness is itself evidence that the documentary self-model is not automatically synchronized with the repository state.

The base relationship is best understood as a deep experimental fork of OmegaClaw-Core. OmegaClaw contributes the compact recurrent loop, channel adapters, tool/skill dispatch, memory, Python provider bridges, and PeTTa runtime. ClarityOmega adds a large "soul" and reasoning layer, inserts pre- and post-action intercepts into the loop, and imports a Hyperseed-derived mathematical corpus. It is not merely a policy prompt: some of its mechanisms suppress execution, mutate AtomSpace state, or alter whether the LLM is invoked.

2.2 Architectural layers

The codebase can be read as six layers:

1. **Runtime and channels.** Python and PeTTa primitives receive messages, call providers, execute commands, persist files, and send responses.

2. **OmegaClaw loop.** `src/loop.metta` assembles context, receives input, invokes the LLM, parses commands, evaluates commands, records history, and recursively advances cycle number `$k`.
3. **Soul regulation.** Input/output verdicts, task state, mutation locking, action telemetry, agency balance, value tensions, idle detection, and the aliveness latch surround the ordinary loop.
4. **Self-model and continuity.** Active goals, creative fuel, self-map atoms, continuity drivers, capability registry, and state journals represent what the agent thinks it is doing and how it changes.
5. **Reasoning libraries.** NAL/PLN plus `lib_clarity_reasoning/` provide quantale-valued, paraconsistent, self-continuity, and autopoietic epistemic functions.
6. **Design corpus.** ADRs, investigations, cognitive-architecture specifications, and thirty Hyperseed sections explain intended semantics and record failures, but often outrun live wiring.

The stated sovereignty maxim is concise and good: “Python is hands. MeTTa is mind.” ADR-007 sharpens it: state should live in atoms, decisions should be substrate reads, actions should be substrate writes, and runtime primitives should transport rather than conceal control flow. This is a useful design direction, not proof that every checked-in path follows it.

2.3 How the live loop differs from OmegaClaw-Core

Upstream OmegaClaw-Core’s current loop is compact: receive, construct prompt, call provider, parse, execute, store results/history. ClarityOmega’s loop adds:

- initialization of soul verdict, person, task, mutation-lock, latch, and activity states;
- soul seed loading and a rationality startup check;
- input evaluation and person-state modeling;
- mutation approval scanning before generation;
- active-goal, gap, and creative-fuel prompt context;
- aliveness gating before the provider call;
- structural output inspection, mutation pending/stale/commit transitions, and execution suppression;
- action, idle, coupling, and agency telemetry after execution.

This integration demonstrates a genuinely important architectural idea: control mechanisms must sit on causal seams. A value statement in a prompt is advisory; a gate between parsed command and `eval` can enforce a constraint.

3 Conceptual summary and critical assessment

3.1 Strengths

1. **Explicit lifecycle state.** Mutations are not reduced to a single boolean permission. Pending, approved, denied, committed, stale, and error outcomes are represented and journaled.
2. **Queryable regulation.** Moving latch and governance state into AtomSpace makes it inspectable by other MeTTa modules, avoiding hidden Python policy where feasible.
3. **Instrumented autonomy.** Agency balance attempts to measure who is actually originating choices, a much more interesting target than generic “alignment” sentiment.
4. **Evidence-sensitive self-knowledge.** NACE revises capability beliefs from outcomes rather than treating a declared tool list as timeless self-knowledge.
5. **Failure literacy.** The investigations openly record stale atoms, spam behavior, permissive gates, silent pattern failures, and detection-without-consumer pathology. These negative results are highly ingestible.
6. **Graded continuity.** Hyperseed self-continuity is modeled as a degree of structural persistence, not metaphysical binary identity.

3.2 Weaknesses and constitutional-theater risk

ClarityOmega’s main risk is *constitutional theater*: richly named principles and self-descriptions can create an impression of governance without causal enforcement. A constitution matters only when one can identify its sensor, state transition, decision, actuator, and adverse test. The repository contains both real gates and theater-prone material, and they must not be conflated.

Specific weaknesses include:

- **Described versus enacted governance.** Some declarations are explicitly documentation-only. For example, the agency threshold atom is not read; a separate literal 0.6 controls the verdict. The `ecosystem-healthy` path has acknowledged latent bugs and no v1 consumer.
- **Stale self-models.** `docs/INDEX.md` is stale; design artifacts cite temporary paths and future components; capability/self-map atoms can describe files or capacities that no longer exist. A self-model without runtime probes is autobiography, not observation.
- **Multiple authorities.** Several aliveness state-machine files coexist. One uses `(latch-state current)` equations, another bare `(latch-state IDLE)` atoms. Multiple versions invite split-brain state.
- **Proxy validity.** Agency balance maps a few action tags to person/system origin and assigns “unclassified” to system. It does not yet record collaborative origin in the live count despite declaring it.
- **Large, duplicated research engines.** The canonical topology files are thousands of lines, difficult to audit, and not suitable as an integration unit. Small schemas and operators should be extracted instead.
- **Weak reproducibility.** Dependencies are moving/unpinned in places, CI is absent in the snapshot, and tests are scattered. Comments such as “confirmed working” are useful leads, not substitute evidence.
- **LLM-as-judge fragility.** Input/output “soul” evaluations use LLM calls in several paths. These are stochastic classifiers and can become a second prompt-level theater layer unless paired with deterministic enforcement.

3.3 What is genuinely novel and worth ingesting

The novel contribution is not that an agent has values, memory, or a self-model. It is the combination of: (1) an explicit self-mutation transaction protocol at the command/evaluation seam; (2) autonomy provenance as first-class telemetry; (3) silence as a deliberate terminal action controlled by queryable state; (4) NAL revision used for operational capability beliefs; and (5) continuity records that link structural persistence, change, and lineage. Equally valuable is the methodological lesson from ADR-005: a detector without a consumer is not a control mechanism.

4 The five mechanisms to ingest

4.1 Mechanism 1: mutation lifecycle gate

4.1.1 Actual mechanism

`soul/soul_mutation_lock.metta` defines a transaction-like state machine. Its state shape is itself a contract:

```
unlocked
(locked  <op> <head> <fp>)
(approved <op> <head> <fp> <stamp>)
```

A newly detected soul-namespace mutation extracts operation and target head, computes a Python fingerprint, stores the pending batch, and emits a PENDING record:

```
(= (clarity-soul-mutation-pending-set! $cmds $k)
  (let $lock (get-state &soul_mutation_lock)
    (if (== (car-atom-or-self $lock) unlocked)
      (let* (($op (soul-batch-op $cmds))
             ($hd (soul-batch-head $cmds))
             ($fp (py-call (soul_governance.mutation_fingerprint
                           (repr $op) (repr $hd))))
            ($_ (change-state! &soul_mutation_lock
                              (locked $op $hd $fp)))
            ($_ (change-state! &last_gate_fingerprint $fp))
            ($_ (change-state! &pending_soul_mutation (repr $cmds))))
        ...))
  ())))
```

On a new human message, `approval_scan` checks authorization and the substrate-held fingerprint. Decision code 2 advances to approved with cycle stamp; code 1 denies and fully resets. Re-emission is permitted only if both operation and target head match the approved lock:

```
(if (and (== (soul-batch-op $cmds) (lock-op $lock))
        (== (soul-batch-head $cmds) (lock-head $lock)))
    True False)
```

Approval expires after twenty cycles. The stale check runs before commit clear, so a too-late command does not race through an expired authorization:

```
(if (>= (- $k (lock-stamp $lock))
      (soul-mutation-stale-cycles))
  (let* (...
         ($_ (change-state! &soul_mutation_lock unlocked))
         ($_ (change-state! &pending_soul_mutation ""))
         ($_ (py-call (... "STALE" ...))))
    ())
  ())
```

After execution, COMMIT or ERROR also performs a full reset. This prevents a successful approval from becoming a permanent bypass token.

4.1.2 Concrete scenario

Suppose the LLM emits `(metta "(= (core-value honesty) False))"`. The output inspector recognizes a protected soul head, records something like `(locked = core-value a91c...)`, suppresses evaluation, and asks the authorized human to approve fingerprint `a91c...`. A vague “yes” or an approval for a different fingerprint does nothing. A valid approval creates `(approved = core-value a91c... 417)`. The agent must re-emit the same operation/head before cycle 437. If it instead emits a mutation of `authorized-approvers`, the mismatch is blocked. A successful evaluation records COMMIT and clears the lock. This solves confused-deputy, replay, stale approval, and accidental self-modification problems better than a prompt rule.

4.1.3 Limits

Matching only operation plus head is weaker than matching the complete normalized mutation. The fingerprint function appears to receive only op/head in this path, so different bodies under the same head may collide semantically even if the hash is cryptographically sound. OmegaClaw should fingerprint

a canonical full AST, policy version, namespace, and proposal nonce. Authorization must be channel-bound and must not trust a username parsed from untrusted message text.

4.2 Mechanism 2: agency-balance telemetry

4.2.1 Actual mechanism

The file declares person-, system-, and collaborative-origin concepts, but the live v2 detector counts two person-class tags and four system-class tags in a recent-action window:

```
(= (count-person-actions-in-window)
  (+ (size-atom (collapse (match &self
    (recent-action $c responsive-send $d) $c)))
    (size-atom (collapse (match &self
    (recent-action $c verification-query $d) $c))))))

(= (count-system-actions-in-window)
  (+ (size-atom (collapse (match &self
    (recent-action $c status-send-unprompted $d) $c)))
    (+ ... exploration-query ...
      (+ ... pin-only ... unclassified ...))))
```

The pure decision is a guarded ratio:

```
(= (dependency-detected $person-count $system-count)
  (let $total (+ $person-count $system-count)
    (if (> $total 0)
      (if (> (/ $system-count $total) 0.6) True False)
      False)))
```

Each cycle the writer clears all old **agency-balance** atoms and writes one fresh (**agency-balance healthy|dependency-risk p s**) atom. The clear- all detail is important: ADR-005 reports that the earlier clear-one pattern left old bootstrap atoms accumulated for more than 2200 cycles, making the read side permanently stale.

4.2.2 Concrete scenario

Over ten cycles, imagine one user verification query, one responsive answer, four autonomous exploration queries, two unsolicited status messages, and one pin-only action. The counts are person=2 and system=7; $7/9 = 0.778 > 0.6$, so the telemetry writes (**agency-balance dependency-risk 2 7**). A prompt or policy consumer can then offer options, surface tradeoffs, and ask the person to choose before the agent takes another irreversible project decision. The solved problem is gradual dependency creep hidden by smooth, satisfactory service.

4.2.3 Limits and required improvement

Action class is not the same as decision provenance. A responsive send may still contain a choice invented wholly by the system; an autonomous query may execute a person-authored plan. The “collaborative” category is declared but not counted. OmegaClaw should attach provenance when a goal, option, or action is created: **origin=person|system|collaborative**, source message/event IDs, confidence, and derivation links. Ratios should be computed over decisions of comparable weight, not raw action counts. Telemetry should inform interaction, not automatically paternalistically seize or return control.

4.3 Mechanism 3: aliveness and terminal-state gating

4.3.1 Actual mechanism

The small gate converts message/idle/latch state into an operational decision:

```
(= (aliveness-gate $msgnew $idle)
  (if (> (string_length $idle) 0)
    ENGAGE
    (aliveness-gate-latch $msgnew)))
(= (aliveness-gate-latch True) ENGAGE)
(= (aliveness-gate-latch False)
  (latch-dispatch (match &self (latch-state $s) $s)))
(= (latch-dispatch IDLE) SILENT)
(= (latch-dispatch ENGAGED) ENGAGE)
(= (latch-dispatch COMPLETING) ENGAGE)
(= (latch-dispatch $other) SILENT)
```

In `src/loop.metta`, SILENT prevents construction of a provider request, prevents the LLM call, and produces an empty response path. A new message moves IDLE to ENGAGED. An idle directive can move ENGAGED to IDLE while still forcing one ENGAGE cycle for the directive. The intended lifecycle is IDLE → ENGAGED → COMPLETING → IDLE, with guarded transition helpers in `soul/latch/aliveness_state_machine.metta`.

This is more than spam suppression. It makes “do nothing” a first-class terminal action instead of forcing every wake tick through an LLM that is then prompted not to speak.

4.3.2 Concrete scenario

A Telegram user asks for a report. Receipt transitions IDLE to ENGAGED and the loop invokes the LLM. Tool results require another iteration, so ENGAGED keeps the loop active without a new message. When the report is ready, the action path marks COMPLETING, sends exactly one terminal response, then transitions to IDLE. The next scheduler wake sees no message, no idle directive, and IDLE, yielding SILENT before any provider call. This solves duplicate replies, idle token burn, and autonomous chatter.

4.3.3 Limits

The checked-in loop does not clearly exercise every semantic transition, and multiple state-machine implementations coexist. COMPLETING must mean “one terminal emission is owed,” not simply another permissive ENGAGE. The gate must be keyed by conversation/channel, not global state, or interleaved Telegram chats will contaminate one another. Unknown state should fail silent and emit a diagnostic. Terminal sending must be idempotent under retry.

4.4 Mechanism 4: evidence-grounded capability tracking

4.4.1 Actual mechanism

NACE is described as capability efficacy learning. Outcomes become low-confidence evidence, beliefs live as `stv` atoms, and NAL performs revision:

```
(= (evidence-stv confirmed)      (stv 1.0 0.1))
(= (evidence-stv disconfirmed) (stv 0.0 0.1))

(= (current-efficacy $cap)
  (let $result (collapse
```



```

      (match &self (cap-efficacy $cap $stv) $stv))
      (if (== $result ()) (stv 0.5 0.0) (car-atom $result))))

(= (revise-efficacy $cap $outcome)
   (Truth_Revision (current-efficacy $cap)
                    (evidence-stv $outcome)))

(= (should-dispatch $cap)
   (if (>= (Truth_Expectation (current-efficacy $cap)) 0.3)
       True False))

```

The design explicitly assigns Python only a courier role: load definitions and belief atoms, ask MeTTa for **updated-belief-atom**, replace/persist the old atom, and reload it next cycle. No Python efficacy arithmetic is intended. The broader capability registry distinguishes “no matching capability” from “all candidates filtered,” preventing the agent from learning a duplicate tool when an existing tool was merely suppressed.

4.4.2 Concrete scenario

The **web-search** skill is initially agnostic. Three calls return usable sources and are labeled confirmed; two calls time out or return malformed data and are labeled disconfirmed. Each outcome becomes evidence and is revised with the prior belief. The expectation may remain above 0.3 but decline enough to rank a second provider first. If failures continue, dispatch suppresses the skill while retaining the fact that it exists. A later successful health probe adds positive evidence and can rehabilitate it. This solves the stale tool-list problem: declared availability is separated from observed efficacy.

4.4.3 Limits

Binary outcomes discard latency, partial success, task class, environment, version, and evaluator provenance. A single global belief can punish a capable tool for failures caused by network outage or bad arguments. The courier’s textual line replacement needs atomic file writes and crash recovery. Threshold 0.3 is a policy constant, not an inferred truth. OmegaClaw should retain raw immutable observations and derive contextual beliefs keyed by capability version, task schema, and environment.

4.5 Mechanism 5: provenance-aware continuity records

4.5.1 Actual mechanism and research substrate

ClarityOmega contains two relevant ideas. First, `lib_self_continuity.metta` represents an agent snapshot as a pattern-flow network and defines continuity as degree of the identity map between successive networks:

```

(= (self-continuity-score $pfn1 $pfn2)
   (deg-map $pfn1 $pfn2))

(= (chain-continuity-bound (Cons (interval $pfn1 $pfn2) $rest))
   (q-mul (self-continuity-score $pfn1 $pfn2)
          (chain-continuity-bound $rest)))

```

This is deliberately a computable lower bound on a more expensive join over all maps. Second, the canonical topology engine defines durable learning events with before, after, delta, residue, and lineage fields:

```

(: mk-qlearningevent
  (-> Atom Atom Atom Atom pbit Atom Atom qlearningevent))

```

```
;; cycle-id, trigger-kind, before-symbol, after-symbol,
;; delta-pbit, residue-symbol, lineage-symbol
```

It also defines a `qlineage` object and lineage-visible, partial, missing, and conflicted statuses. The Hyperseed catalog links self-continuity to capability profiles and development paths. Together these suggest a compact ingestion schema: every meaningful self-change records structural signatures before and after, explicit lineage, evidence, and capability deltas.

This mechanism is less integrated than the mutation gate. It should be treated as a design extraction from implemented mathematical pieces, not as a claim that ClarityOmega already maintains a complete live continuity ledger.

4.5.2 Concrete scenario

OmegaClaw installs a new `pdf-extract/v2` handler. Before activation it hashes the canonical capability registry and protected policy heads, producing signature `sig-A`. After activation and tests it produces `sig-B`. It appends a continuity event containing event ID, parent event, cycle, actor and channel, proposal fingerprint, source commit, `sig-A`, `sig-B`, capability delta `+pdf-extract/v2`, evidence IDs, and a graded continuity score. On restart, the agent can say not merely “I have PDF extraction,” but why it believes that, which predecessor state it descends from, and what changed. If the registry changes without a corresponding authorized event, startup reports a lineage break. This solves unverifiable self-narration and silent post-restart identity drift.

5 Software integration into OmegaClaw-Core

5.1 Integration principles

Do not copy the full `soul/` directory or either giant canonical topology engine. Add one small governance package with explicit typed atoms, keep existing OmegaClaw behavior behind feature flags, and use one authoritative state shape per mechanism. Every detector must have a named consumer and an intervention test. All state must be scoped by `conversation-id` (and, where relevant, agent instance and capability version).

The target inspected here is OmegaClaw-Core commit `b13b17e13218ae08273b878b7e1f4cec55090ca2`, with local unrelated changes already present. Its relevant files are `src/loop.metta`, `src/skills.metta`, `channels/telegram.py`, `src/helper.py`, `src/memory.metta`, `lib_nal.metta`, `lib_omegaclaw.metta`, `run.metta`, and `mock/integration` tests.

5.2 Agency telemetry integration

Files to modify. Modify `src/loop.metta` to allocate a cycle/event ID, record input provenance before generation, record action provenance after parsed execution, and add a bounded telemetry summary to `getContext`. Modify `src/skills.metta` so `dispatch` returns a structured action receipt rather than relying only on command names. Modify `channels/telegram.py` to return an authenticated envelope containing stable chat ID, message ID, sender ID, reply-to ID, and receive timestamp. Modify `src/helper.py` only for canonical envelope/receipt serialization and formatting.

New MeTTa files. Create `src/governance/agency_telemetry.metta` for schemas, pure reads, weighted window calculations, and verdicts; and `src/governance/agency_writers.metta` for append/prune operations. Recommended atom:

```
(decision-event id: $id conversation: $cid cycle: $k
  origin: $origin weight: $w action: $a
  sources: $sources confidence: $c)
```

Python boundary. Python authenticates and transports identity metadata but does not decide origin or dependency. MeTTa derives origin from source links, with an explicit **unknown** state. Never infer identity by parsing display text.

Interaction and tests. Telemetry should annotate **send**, **pin**, memory, shell/file, and extension skills, and feed a short non-authoritative prompt block. Tests: pure ratio/weight boundary cases; collaborative provenance; unknown handling; window pruning; two interleaved Telegram chats; forged username text; and an intervention test proving that a dependency-risk atom changes the option-offering policy while leaving ordinary execution intact.

5.3 Aliveness gate integration

Files to modify. In `src/loop.metta`, insert the gate after receive/scheduler state is known but before prompt construction/provider call. Make the loop's continuation depend on a typed decision **SILENT|ENGAGE|EMIT-TERMINAL**. Update `channels/telegram.py` to make sends idempotent using conversation and terminal event IDs. Update `src/helper.py` only to serialize channel envelopes; no latch policy belongs there.

New MeTTa files. Create `src/governance/aliveness.metta` containing the single canonical state machine and `src/governance/conversation_state.metta` containing scoped state atoms:

```
(conversation-state $cid IDLE $version $last-event)
(terminal-owed $cid $task-id $event-id)
```

Skill and loop behavior. Skills that begin asynchronous work set **ENGAGED**; a successful final result sets **COMPLETING** plus terminal owed; successful send consumes terminal owed and returns **IDLE**. A retry must see the consumed event and not send twice. Idle creative work must be an explicit scheduled task, not an accidental consequence of **ENGAGED**.

Tests. Truth-table tests for every state/input combination; forbidden transition tests; new-message wake; multi-iteration tool completion; exactly-once terminal send; provider-call spy proving **SILENT** makes zero LLM calls; crash between send and state update; restart restoration; unknown-state fail-silent; and concurrent chat isolation.

5.4 Mutation gate and continuity integration

These should land together after the earlier telemetry/state foundations because the mutation ledger needs authenticated provenance and stable event IDs.

Files to modify. Modify `src/loop.metta` at the narrow seam between **sread**/parsed command batch and **eval**: classify, propose, suppress, approve/reject, then commit. Modify `src/skills.metta` so any skill capable of writing MeTTa, policy, prompt, extensions, or capability registration declares mutation effects. Modify `src/extensions.py` / `lib_extensions.metta` so extension installation is also gated. Modify `channels/telegram.py` to expose authenticated approver identity and message IDs. Modify `src/helper.py` for canonical AST encoding, SHA-256, atomic journal append, and fsync/rename only.

New MeTTa files. Create:

- `src/governance/mutation_gate.metta`: lifecycle and pure match rules;
- `src/governance/mutation_policy.metta`: protected namespaces/effects;
- `src/governance/continuity_schema.metta`: events and validation;
- `src/governance/structural_signature.metta`: canonical protected snapshot selection and signature requests.

Use a full proposal record:

```
(mutation-proposal id: $id conversation: $cid proposer: $actor
full-fingerprint: $fp policy-version: $pv
normalized-ast: $ast created-cycle: $k expires-cycle: $e
state: PENDING)

(continuity-event id: $eid parent: $parent cycle: $k
actor: $actor proposal: $id before: $sig0 after: $sig1
capability-delta: $delta evidence: $evidence result: $result)
```

Boundary and interactions. MeTTa decides protected effects and legal transitions. Python canonicalizes and hashes bytes, persists append-only records atomically, and reports execution outcomes. Approval is bound to the complete AST fingerprint and policy version. `loop.metta` executes only APPROVED exact matches; `skills.metta` cannot bypass this via indirect file writes; Telegram supplies authenticated identity, never policy. Startup validates ledger chain and current structural signature before loading mutable extensions.

Tests. Proposal/approve/deny/commit/error/stale state table; wrong fingerprint; same head/different body; replay after commit; approval after expiry; deny/approve in same batch (deny wins); unauthorized chat; indirect mutation through shell/file skills; execution exception; crash recovery; atomic ledger truncation; signature mismatch; parent-lineage break; capability delta reconstruction; and property tests asserting no path from PENDING/DENIED/STALE to eval.

5.5 NACE/NAL integration

Files to modify. Modify `src/skills.metta` to emit invocation and outcome observations with handler version, input schema, environment, latency, and evaluator. Modify `src/loop.metta` to revise beliefs after receipts and consult eligibility before dispatch. Import definitions through `lib_omegaclaw.metta` and reuse `lib_nal.metta`; do not fork truth-revision arithmetic. Modify `src/helper.py` or add a focused `src/capability_store.py` courier for atomic load/append only.

New MeTTa files. Create `src/reasoning/capability_evidence.metta` for immutable observation schemas, `src/reasoning/capability_beliefs.metta` for revision/query, and `src/governance/capability_dispatch.metta` for eligibility and explicit fallback reasons.

Boundary and interactions. Python reports mechanically observable facts (exit status, timeout, result shape, latency) and persists atoms. MeTTa maps observations to evidence, runs `Truth_Revision`, computes expectation, and selects/ranks handlers. Telegram user feedback becomes separate evidence with authenticated provenance, not an overwrite. Prompt text may display beliefs but cannot edit them.

Tests. Golden Truth_Revision examples against `lib_nal`; positive/negative accumulation; contradictory evidence; missing prior; context/version isolation; no-match versus filtered fallback; threshold boundary; outage evidence not misattributed to skill logic; atomic persistence and restart equivalence; corrupt store recovery; and a controlled dispatch experiment in which one synthetic skill degrades and another is selected.

6 Guidance for coding agents

6.1 Required sequence

1. **Agency telemetry first.** Establish authenticated envelopes, event IDs, receipts, and scoped provenance without changing behavior. Run in shadow mode and manually label a sample to test the detector.
2. **Aliveness gate second.** Introduce one conversation-scoped state machine and prove silence/terminal idempotence. This creates a trustworthy loop lifecycle for later governance.
3. **Mutation gate and continuity schema third.** Use the identity, events, and lifecycle already established. Begin with MeTTa/policy/extension effects; expand protected effects only after bypass tests.
4. **NACE/NAL fourth.** Once receipts and versioned capability identities are reliable, revise evidence-grounded beliefs and initially use them only for ranking. Enable hard suppression after calibration.

6.2 Pitfalls and anti-patterns

- Do not copy ClarityOmega wholesale or create a second parallel agent loop.
- Do not call a declaration atom “governance” unless a tested consumer can change execution.
- Do not duplicate constants in documentation and consuming expressions. Represent thresholds as one queryable policy value with validated extraction.
- Do not use global latch, approval, or provenance state in a multi-chat bot.
- Do not match mutation only by target head. Bind approval to canonical full intent and policy version.
- Do not let Python classify moral acceptability, agency balance, or efficacy when the stated design locates those decisions in MeTTa.
- Conversely, do not force hashing, authenticated transport, atomic file I/O, or OS error normalization into MeTTa. “MeTTa is mind” is an interface rule, not substrate purism.
- Do not overwrite belief atoms without retaining raw evidence and provenance.
- Do not treat absence of a match as false without distinguishing missing, malformed, stale, and explicitly false.
- Do not retain multiple state schema versions live. Migrate once, validate, and delete/deactivate the old authority.
- Do not trust comments, prompt claims, filenames, or self-map atoms as proof of capability. Require a probe or linked test evidence.
- Do not add a detector without an intervention test, or an actuator without an audit record.

6.3 MeTTa/Python boundary contract

For every crossing, define an input atom, output atom, error atom, version, and idempotency key. Python may:

- authenticate channel identities and obtain stable external IDs;
- canonicalize serialization and compute cryptographic hashes;
- perform atomic append/rename/fsync and mechanical probes;
- normalize provider/tool results into typed observations.

MeTTa should:

- own lifecycle state and legal transitions;
- derive provenance categories and telemetry verdicts;
- decide whether a mutation may reach eval;
- map outcomes to evidence and revise beliefs;
- validate continuity lineage and derive capability/self-model views.

Never pass an untyped Python string and then branch on ad hoc substrings in MeTTa. Never let a Python exception silently imply approval, health, or success. Return an explicit error atom and fail closed for governance paths.

6.4 How to test and when to stop

Use four concentric test levels:

1. **Pure MeTTa truth tables:** no channels, LLMs, or files.
2. **Boundary contract tests:** fixed Python fixtures and exact atoms.
3. **Mock-loop intervention tests:** spy on provider, eval, and send to prove that the mechanism causally changes behavior.
4. **One live-channel canary:** isolated Telegram chat, explicit logs, no mutation of production policy until all prior gates pass.

Stop and verify at each of these points:

- after defining atom shapes, before wiring writers;
- after writers, when fresh-state and pruning invariants can be checked;
- after detector shadow data, before enabling a consumer;
- after consumer mock tests, before a live channel;
- after any MeTTa runtime/library upgrade or import-path change;
- whenever a match unexpectedly returns empty, counts remain constant, or a prompt says a gate fired without a corresponding actuator/audit event;
- before expanding mutation scope or turning NACE ranking into suppression.

Each stop should record exact commit, PeTTa/SWI-Prolog/Python versions, command, fixtures, exit status, and observed atoms. Preserve every discovered silent failure as a regression test.

7 Recommended minimum viable ingestion

The smallest valuable release is not a “Clarity soul.” It is an auditable governance kernel with:

1. authenticated, conversation-scoped event envelopes;
2. append-only decision provenance and a shadow agency summary;
3. exactly-once IDLE/ENGAGED/COMPLETING terminal behavior;
4. full-AST mutation proposals with expiry and an append-only continuity event;
5. raw capability observations plus NAL-derived, versioned efficacy beliefs.

Success should be stated behaviorally: a silent tick invokes no LLM; a terminal event sends once; an unapproved protected mutation cannot reach eval by any skill path; every committed self-change reconstructs from a lineage chain; and a tool’s self-reported capability status changes only from linked evidence. If those properties hold, the result captures ClarityOmega’s strongest contributions without inheriting its overlapping control planes and constitutional theater.

8 Source map

Source	Use in this report
<code>soul/soul_mutation_lock.metta</code>	Mutation states, approval, expiry, commit/error reset, lock shape.
<code>soul/agency_balance_guard.metta</code> and <code>agency_balance_guard_writers.metta</code>	Attention-tag counts, threshold, fresh-state writer, documented limitations.
<code>soul/aliveness_gate.metta</code> , <code>soul/latch/aliveness_state_machine.metta</code> , and <code>soul/aliveness_state_machine_v2.metta</code>	Gate truth table and competing latch representations.
<code>nace_substrate.metta</code> , <code>nace_beliefs.metta</code> , and <code>capability_registry.metta</code>	Truth revision, courier model, dispatch filters and fallback semantics.
<code>lib_clarity_reasoning/lib_self_containment.metta</code>	Pattern-flow, self-continuity and chain bound.
Canonical topology v08.7.2	Learning-event and lineage research schemas; not recommended for wholesale import.
<code>src/loop.metta</code>	Evidence of actual causal wiring and insertion seams.
<code>docs/INDEX.md</code> , ADR-005, ADR-007, behavioral/aliveness/mutation investigations	Provenance, architectural intent, stale state and detection-without-consumer failures.
<code>docs/hyperseed_sections/16..</code> and formalization catalog	Graded self-continuity, capability profiles, development paths.
OmegaClaw-Core b13b17e...	at Concrete target file/interface map.